How does a network remember the past?

$$\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T \rightarrow \mathbf{h}_t \rightarrow \mathbf{c}_t \rightarrow \mathbf{e}_w \rightarrow \alpha_{\tau t} \rightarrow \hat{y}$$

Roadmap

1. Sequences as data — windows, splits and the baseline to beat
2. Recurrence and its gradients — one cell, many steps
3. Memory that lasts — LSTM and GRU
4. Text as a sequence — tokens, embeddings, order
5. Sequence to sequence — and the first form of attention

Practice A after Part 3; Practice B after Part 5.

From fixed inputs to sequences

Sessions 03 and 04: the **architecture** and the **objective** shape the representation $\mathbf{h} = g(\mathbf{x})$.

So far every input had a **fixed size**: 28×28, 32×32, 64×64.

Now the length changes, and the **order** of the steps carries the information.

The question for today

> **How does a network remember the past?**

It sees one step at a time.

- What should it keep from the steps before?
- How does the gradient reach a step that happened long ago?

Part 1 · Sequences as data

1. **Sequences as data** — windows, splits and the baseline to beat
2. Recurrence and its gradients — one cell, many steps
3. Memory that lasts — LSTM and GRU
4. Text as a sequence — tokens, embeddings, order
5. Sequence to sequence — and the first form of attention

> Session 03: the architecture is an assumption about the input. Today the assumption is about **time**.

Order matters

Text, sensors, audio, prices, gait, heart rate.

Shuffle the steps and the meaning can disappear: "not good, but fun" ≠ "good, but not fun".

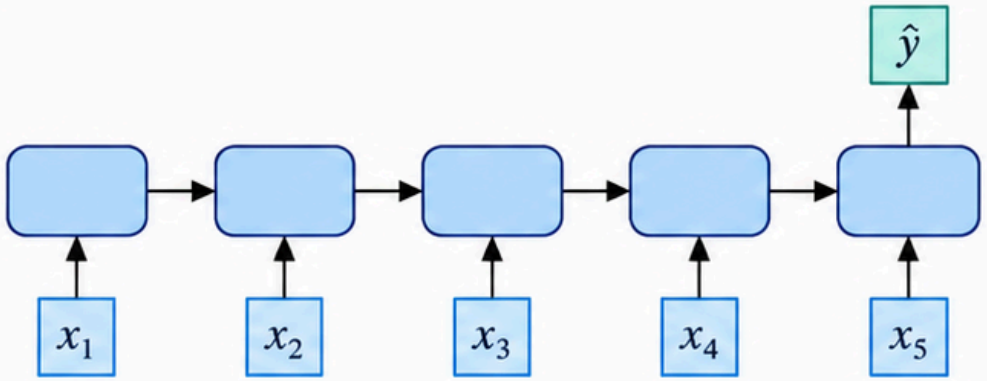
Session 03: shuffling the pixels hurt the CNN, not the MLP. An architecture that ignores order **cannot** use it.

Tasks on sequences

Task	Input → output	Example
many-to-one	sequence → one value	sentiment of a review, next temperature
aligned many-to-many	one output per step	tagging each word, anomaly per reading
sequence-to-sequence	sequence → another sequence	translation, date conversion
one-to-many	one input → sequence	generating text from a start

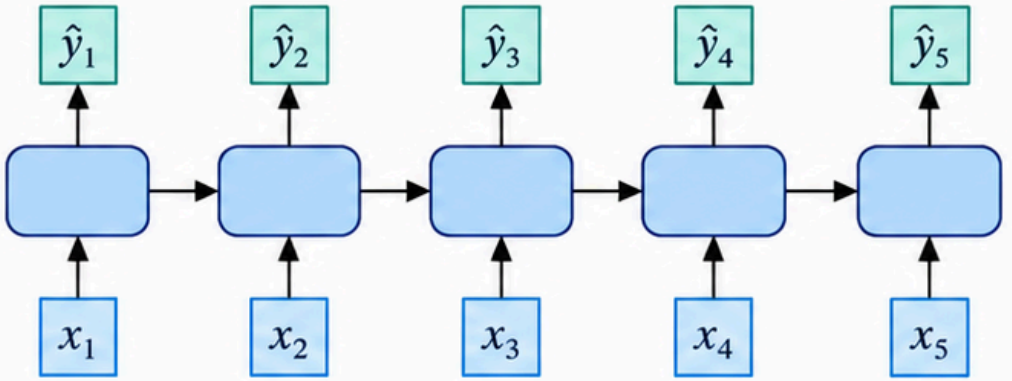
Today: many-to-one in Parts 1–4, sequence-to-sequence in Part 5.

many-to-one



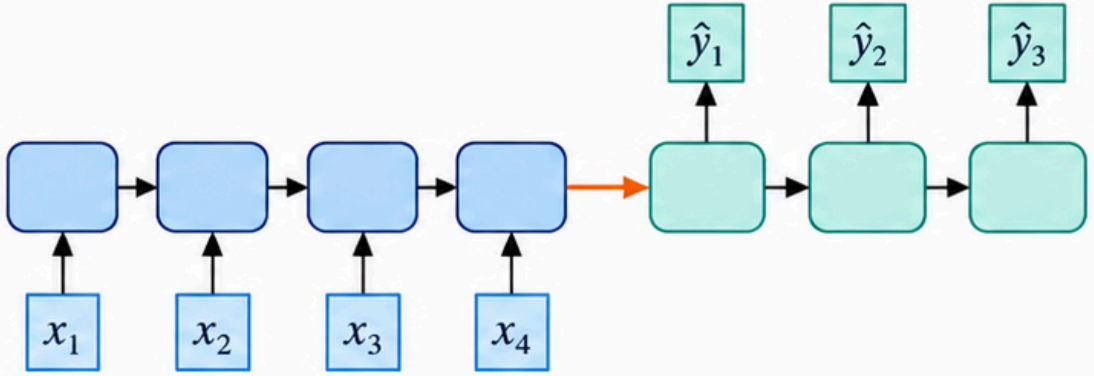
sentiment of a review • temperature in 24 h

aligned many-to-many



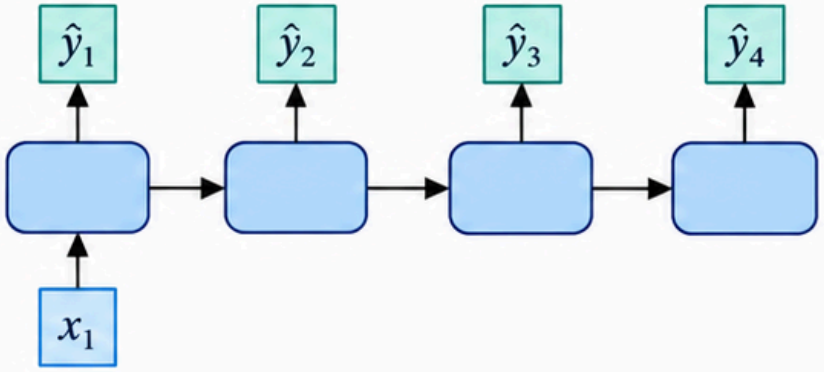
a tag for every word • an anomaly flag for every reading

sequence-to-sequence



translation • date in words → ISO date

one-to-many



generating text from a start

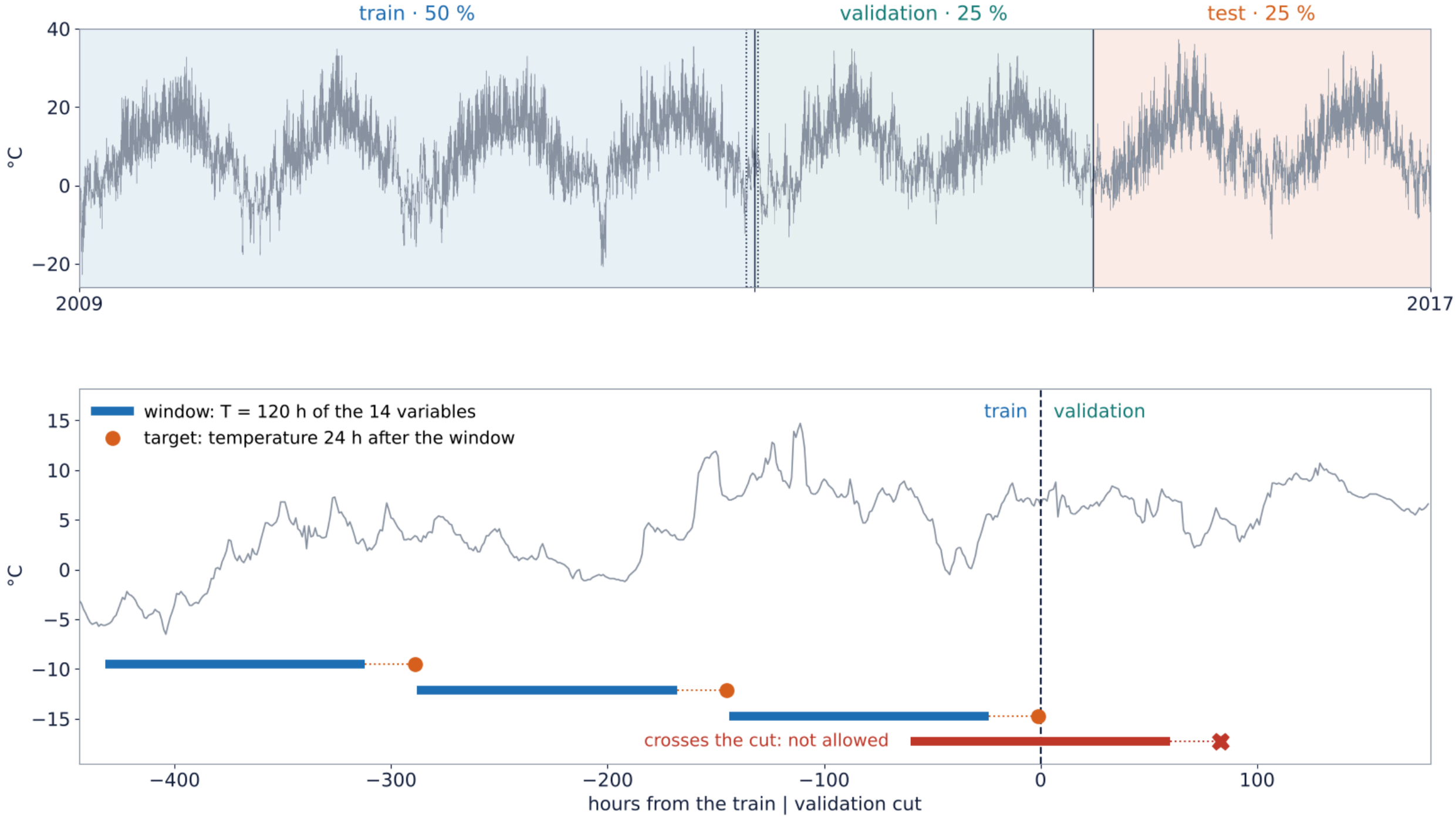
From a series to examples

Jena Climate: 14 variables (temperature, pressure, humidity, wind…) every **10 minutes**, 2009–2016.

A **sliding window** turns one long series into many examples:

$$\underbrace{\mathbf{x}_{t-T+1}, \ldots, \mathbf{x}_t}_{T \text{ past steps}} \longmapsto \; y = \text{temperature at } t + H_{\text{fut}}$$

Notebook 5.1: one reading per hour, $T = 120$ hours (5 days), $H_{\text{fut}} = 24$ hours.


train · 50 %
validation · 25 %
test · 25 %
40
20
0
−20
°C
2009
2017
window: T = 120 h of the 14 variables
target: temperature 24 h after the window
train
validation
15
10
5
0
−5
−10
−15
°C
crosses the cut: not allowed
−400
−300
−200
−100
0
100
hours from the train | validation cut

Three ways to leak the future

1. **Shuffle before splitting.** Neighboring windows overlap almost completely; one lands in train, its twin in validation.
2. **Normalize with the whole series.** The mean and the scale of the test years enter the training data.
3. **Windows that cross a cut.** The input lives in train, the target in validation.

Each one gives a validation score the test will not repeat.

> The data contract of the S02–S04 integrators, now with time in it: **fit everything on the past only.**

Prediction before experiment

Jena, 5 days of hourly readings → temperature in 24 h.

Commonsense baseline: the temperature in 24 h equals the temperature now.

Flattened MLP: $120 \times 14 = 1680$ inputs, two hidden layers.

Which one has the lower validation MAE? Answer in notebook 5.1.

Why not an MLP or a CNN?

Flattened MLP: parameters grow with T; nothing is shared across time; order is only a position in a vector.

1-D convolution (Session 03 along time): shares weights, but sees a fixed receptive field.

What we want:

- the **same weights at every step**;
- a **summary** of everything seen so far, of fixed size.

Part 2 · Recurrence and its gradients

1. Sequences as data — windows, splits and the baseline to beat
2. **Recurrence and its gradients** — one cell, many steps
3. Memory that lasts — LSTM and GRU
4. Text as a sequence — tokens, embeddings, order
5. Sequence to sequence — and the first form of attention

> Part 1: a series becomes windows, split by time; the baseline sets the bar.

The recurrent cell

$$\mathbf{h}_t = \varphi\big(W_{xh}\,\mathbf{x}_t + W_{hh}\,\mathbf{h}_{t-1} + \mathbf{b}\big), \qquad \mathbf{h}_0 = \mathbf{0}$$

- the **same** W_{xh}, W_{hh}, $\mathbf{b}$ at every step: weight sharing along time;
- $\mathbf{h}_t$ summarizes $\mathbf{x}_{1:t}$ in a vector of fixed size;
- many-to-one: $\hat{y} = \mathbf{w}^\top \mathbf{h}_T + b_{\text{out}}$.

Parameters do not depend on T.

Unrolling through time

S05-A01 · Unrolling and backpropagation through time

The cell rolled up, then unrolled over 8 steps with the same W_{hh} highlighted at every step; the gradient flows back from step 8 and its arrow thins step by step.

`../figures/s05_a01_unrolling_bptt.mp4` · route: Manim (`escenas_s05.py`)

Shapes in PyTorch

```
rnn = nn.RNN(input_size=14, hidden_size=16, batch_first=True)
out, h_T = rnn(x)           # x: (B, T, 14)
# out: (B, T, 16)  every h_t      h_T: (1, B, 16)  the last one
y_hat = head(out[:, -1])   # many-to-one: read h_T
```

Notebook 5.2 writes the cell as a Python loop over t and checks that it gives the same `out` as `nn.RNN`.

Backpropagation through time

The unrolled graph is a **deep network with tied weights**: one layer per step.

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} = \prod_{k=t+1}^{T} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}} = \prod_{k=t+1}^{T} \operatorname{diag}\big(\varphi'(\mathbf{z}_k)\big)\, W_{hh}$$

Session 02: depth was a product of Jacobians. Here it is the **same matrix**, $T - t$ times.

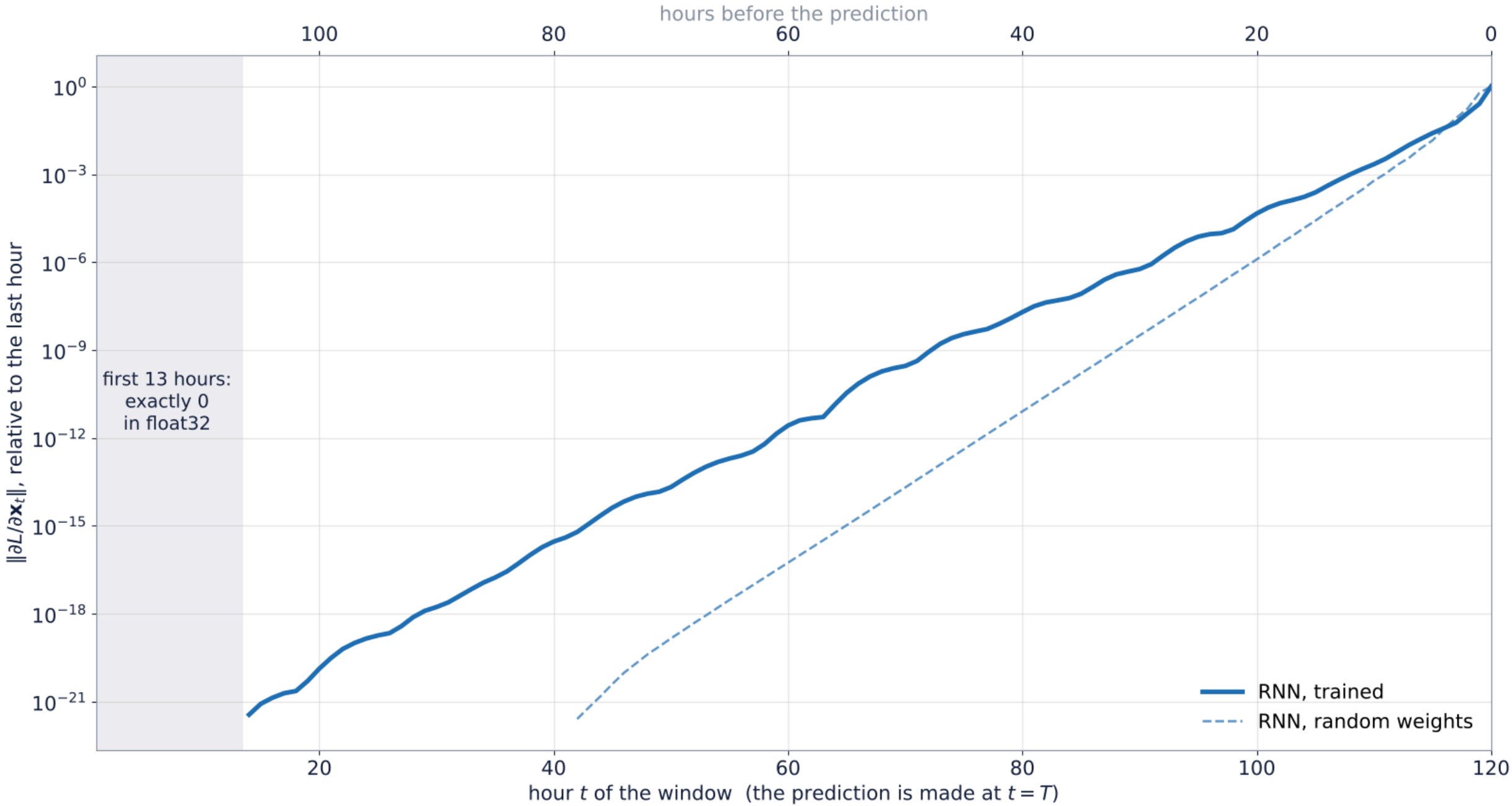
Vanishing and exploding

$$\left\| \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} \right\| \leq \left(\gamma \, \| W_{hh} \| \right)^{T-t}, \qquad \gamma = \max |\varphi'|$$

- $\gamma \| W_{hh} \| < 1$: the signal from step t **fades geometrically**;
- large $\| W_{hh} \|$: the product can **explode**.

With tanh, $\gamma = 1$ and $\varphi' < 1$ away from zero: fading is the usual case.

Bengio, Simard & Frasconi (1994); Pascanu, Mikolov & Bengio (2013).


hours before the prediction
100
80
60
40
20
0
10^{0}
10^{-3}
10^{-6}
10^{-9}
10^{-12}
10^{-15}
10^{-18}
10^{-21}
$\|\partial L/\partial \mathbf{x}_t\|$, relative to the last hour
first 13 hours:
exactly 0
in float32
RNN, trained
RNN, random weights
20
40
60
80
100
120
hour t of the window (the prediction is made at $t = T$)

Prediction before experiment

Same Jena windows, a recurrent network with 16 units.

Does it beat the **commonsense baseline**?
And the flattened MLP?

With a window of **1 day** instead of 5, does it get better, worse, or stay the same?

Answer in notebook 5.2.

Gradient clipping

If $\|\nabla_\theta L\| > c$, rescale it to norm c:

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

It stops **explosions**. It does nothing for **fading**: a small gradient stays small.

Session 02: read the gradient norm before choosing a fix.

Truncated BPTT

For very long sequences, backpropagate only k steps and carry the state forward **detached**:

```
h = h.detach()   # the next chunk starts from h, but no gradient flows back into it
```

Cheaper in memory, and it limits **by design** how far back the network can learn.

What we need

A path along time where the gradient is **not** multiplied by W_{hh} at every step.

Session 02 had the answer for depth:

$$\mathbf{x}_{l+1} = \mathbf{x}_l + F(\mathbf{x}_l) \qquad \Rightarrow \qquad \frac{\partial \mathbf{x}_{l+1}}{\partial \mathbf{x}_l} = I + \frac{\partial F}{\partial \mathbf{x}_l}$$

An **additive** path. Part 3 puts one along time.

Part 3 · Memory that lasts: LSTM and GRU

1. Sequences as data — windows, splits and the baseline to beat
2. Recurrence and its gradients — one cell, many steps
3. **Memory that lasts** — LSTM and GRU
4. Text as a sequence — tokens, embeddings, order
5. Sequence to sequence — and the first form of attention

> Part 2: the repeated product of Jacobians wipes out the past.

An additive path for memory

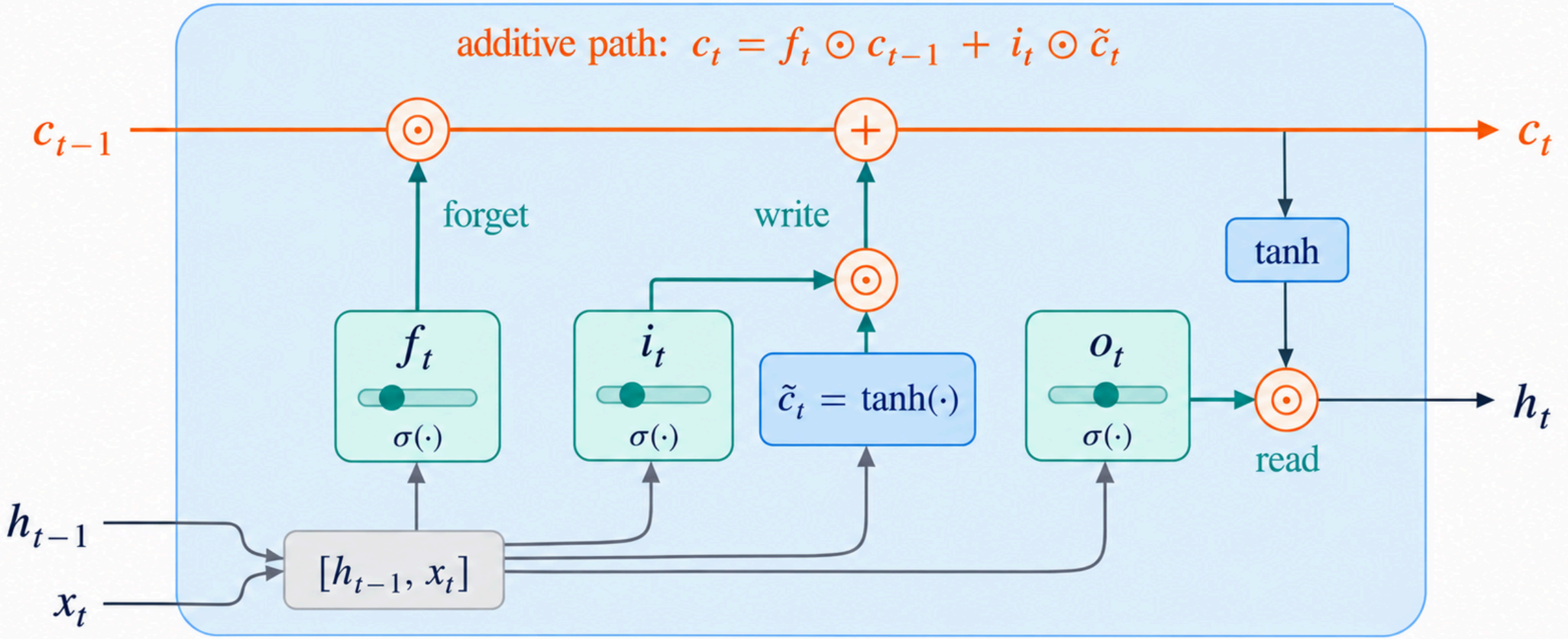
The LSTM keeps a second state, the **cell** $\mathbf{c}_t$:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \qquad \tilde{\mathbf{c}}_t = \tanh\left(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c\right)$$

Addition instead of multiplication by W_{hh}: the residual of Session 02, **along time**.

With $\mathbf{f}_t \approx \mathbf{1}$: $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} \approx I$.

Hochreiter & Schmidhuber (1997); forget gate: Gers, Schmidhuber & Cummins (2000).



with $f_t \approx 1$: $\dfrac{\partial c_t}{\partial c_{t-1}} \approx I$ — the residual of Session 02, along time

Three gates

$$\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t = \sigma\big(W_{f,i,o}\,[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_{f,i,o}\big) \in (0,1)^{d_h}$$

Gate	Decides
forget $\mathbf{f}_t$	how much of $\mathbf{c}_{t-1}$ to keep
input $\mathbf{i}_t$	how much of the new candidate to write
output $\mathbf{o}_t$	how much of the memory to show: $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$

A gate is a **learned multiplier that depends on the input**. Common trick: forget bias initialized to 1, so memory is kept from the start.

The memory in motion

S05-A02 · Writing, keeping and reading a memory

A value written into $\mathbf{c}_t$ at step 2, kept by $\mathbf{f}_t \approx 1$, read by $\mathbf{o}_t$ at step 7. The same input in a plain RNN fades step after step.

`../figures/s05_a02_lstm_memory.mp4` · route: Manim (`escenas_s05.py`)

GRU

Two gates, no separate cell (Cho et al., 2014):

$$\mathbf{h}_t = (1 - \mathbf{u}_t) \odot \mathbf{h}_{t-1} + \mathbf{u}_t \odot \tilde{\mathbf{h}}_t$$

- update gate $\mathbf{u}_t$: interpolates old and new; the additive path is still there;
- reset gate $\mathbf{r}_t$: how much of $\mathbf{h}_{t-1}$ enters the candidate $\tilde{\mathbf{h}}_t$.

Three blocks of weights instead of four: about **3/4** of the LSTM parameters. Often similar results; choose on validation, not by habit.

Prediction before experiment

Notebook 5.3: LSTM and GRU on the same Jena windows, same hidden size, same budget.

Do they beat the RNN and the baseline? By how much?

Does the gradient-vs-time plot of F03 change **shape**, or only height?

Stacked and bidirectional

Stacked: $\mathbf{h}_t$ of layer 1 is $\mathbf{x}_t$ of layer 2. `nn.LSTM(..., num_layers=2)`.

Bidirectional: one pass forward, one backward, states concatenated. `bidirectional=True`.

Bidirectional reads the **whole input** before predicting:

- text classification: fine, the review is complete;
- forecasting: fine *inside* the window, never beyond its end.

Regularizing recurrent networks

- dropout on the inputs and outputs of the cell;
- dropout on the recurrent connection with the **same mask at every step** (Gal & Ghahramani, 2016); a new mask per step destroys the memory;
- weight decay and early stopping, as in Session 02;
- gradient clipping, always on.

`nn.LSTM(dropout=p)` applies dropout only between stacked layers, not on the recurrent connection.

Practice A — Recurrent networks on Jena

Worked notebooks, one change per cell, GPU recommended:

- 5.1 · Series and windows: the three leaks, shown and fixed; baseline; flattened MLP; 1-D CNN
- 5.2 · RNN and BPTT: the cell as a loop, gradient vs t, clipping, 1 day vs 5 days
- 5.3 · LSTM and GRU: same windows, same plot, stacking, recurrent dropout

> Keep one table: model, parameters, validation MAE, MAE minus baseline.

Part 4 · Text as a sequence

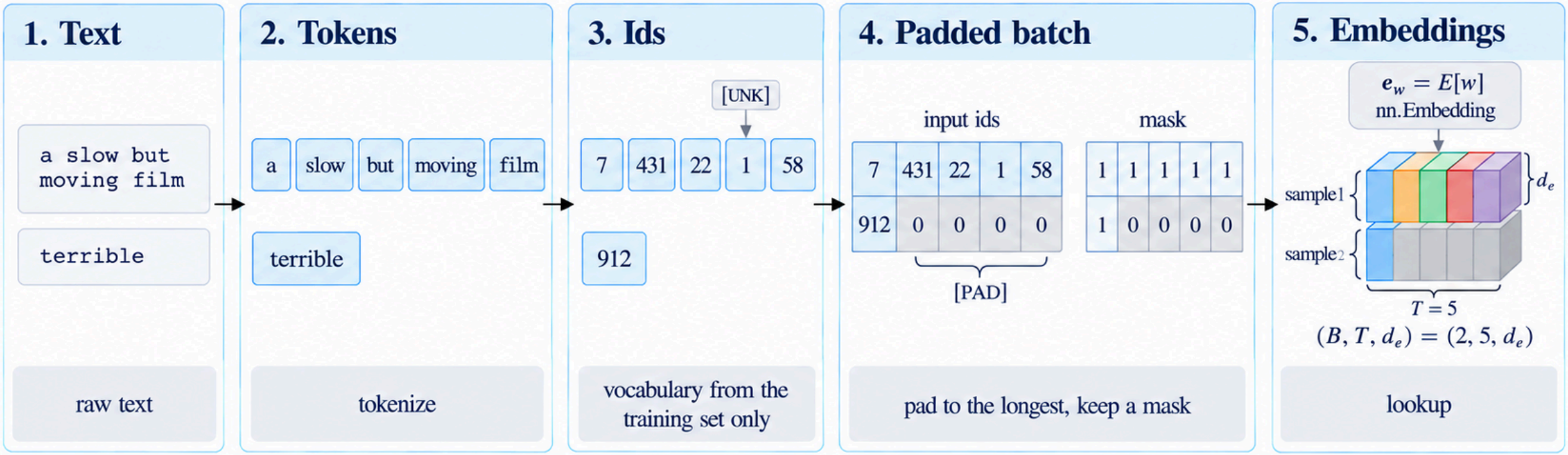
1. Sequences as data — windows, splits and the baseline to beat
2. Recurrence and its gradients — one cell, many steps
3. Memory that lasts — LSTM and GRU
4. **Text as a sequence** — tokens, embeddings, order
5. Sequence to sequence — and the first form of attention

> Part 3: an additive path keeps the signal over long windows. Now the steps are **words**.

From text to integers

1. **Tokenize**: split the text into units.
2. **Vocabulary**: the most frequent tokens of the **training set** only; the rest map to `[UNK]`.
3. **Ids**: each token → an integer; special tokens `[PAD]`, `[UNK]`.

Unit	Vocabulary	Sequence length	Unknown words
characters	tiny	very long	none
words	large	short	many
subwords (BPE)	chosen	in between	none — Session 06


1. Text
a slow but moving film
terrible
raw text
2. Tokens
a
slow
but
moving
film
terrible
tokenize
3. Ids
[UNK]
7
431
22
1
58
912
vocabulary from the training set only
4. Padded batch
input ids
mask
7
431
22
1
58
912
0
0
0
0
1
1
1
1
1
1
0
0
0
0
[PAD]
pad to the longest, keep a mask
5. Embeddings
$e_w = E[w]$
nn.Embedding
sample1
sample2
d_e
$T = 5$
$(B, T, d_e) = (2, 5, d_e)$
lookup

Embeddings

$$\mathbf{e}_w = E[w], \qquad E \in \mathbb{R}^{|V| \times d_e}$$

`nn.Embedding(|V|, d_e)`: one-hot × matrix, done as a lookup (Session 04).

- learned **with the task**, from random;
- or initialized from word2vec or GloVe, trained on other text.

Distance in E comes to mean "similar **for this task**".

Batches of different lengths

Reviews have 10 words or 1 000.

- **pad** to the longest in the batch with `[PAD]`, and keep a **mask**;
- the model must ignore the padding: masked mean, or `pack_padded_sequence` for the LSTM;
- **truncate** long texts to a maximum length: a design choice to report;
- group similar lengths in a batch to waste less compute.

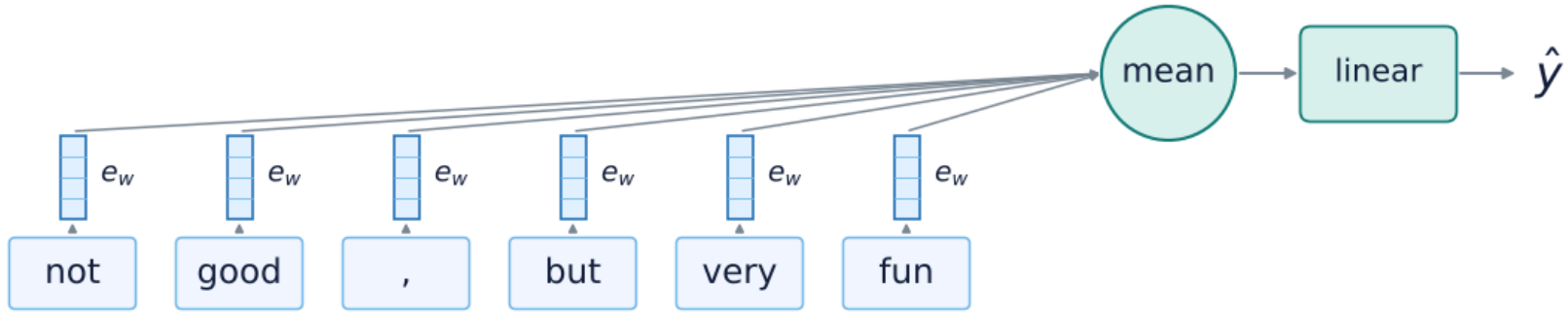
A classic silent bug: averaging embeddings including the padding. Short reviews get pulled toward the `[PAD]` vector.

Three classifiers, same budget

Model	Sees order?	Reads
bag of embeddings	no	$\frac{1}{T}\sum_t \mathbf{e}_{w_t}$ → linear layer
1-D CNN over embeddings	locally, k words	filters over windows of k words, max over time
LSTM	all of it	$\mathbf{h}_T$ (or a pooling of every $\mathbf{h}_t$)

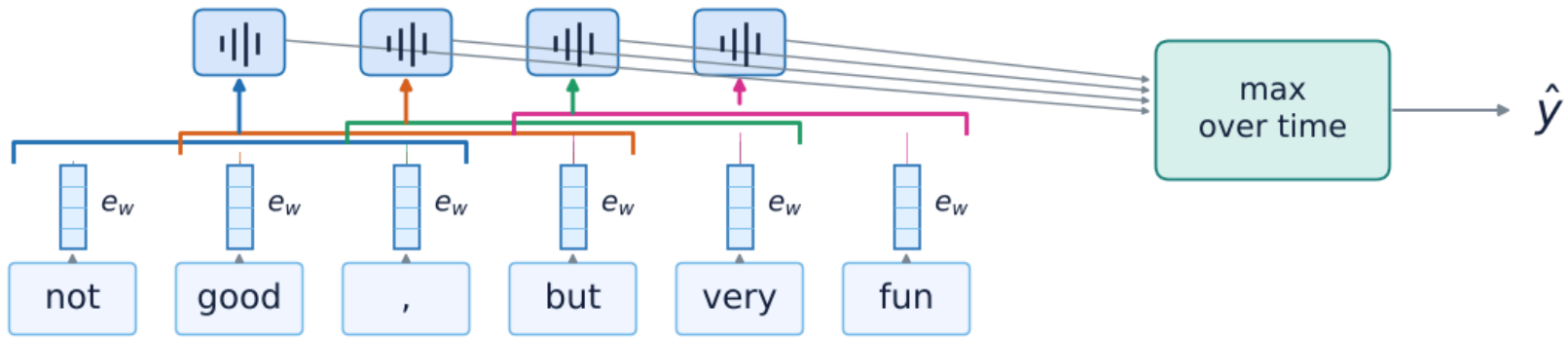
Same embedding size, same training budget, IMDB (25 000 reviews for training, 25 000 for test).

bag of embeddings



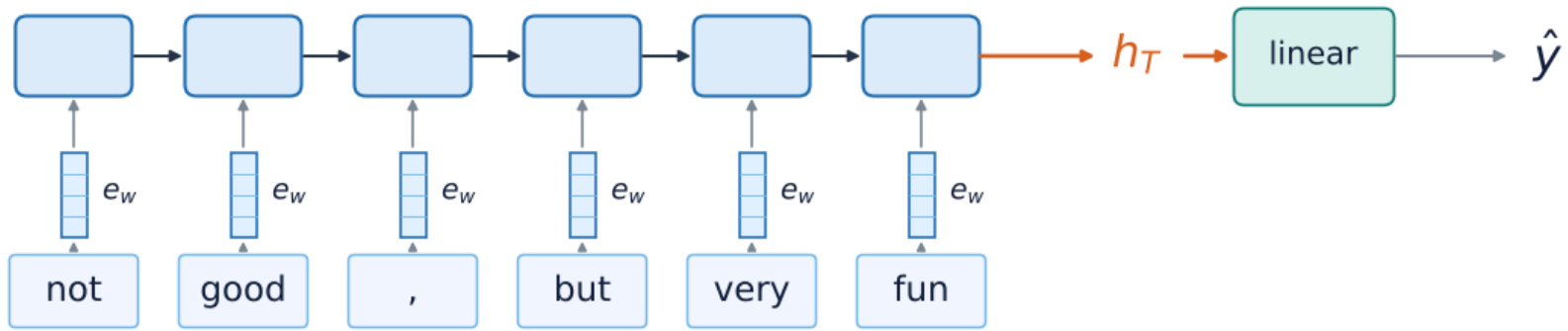
same output for any order of the words

1-D CNN



sees order within 3 words

LSTM



sees the whole order, step by step

What the embedding learned

After training on sentiment, look at the nearest neighbors of a few words in E (cosine similarity, as in Session 04).

The neighbors are grouped by **polarity**, not by meaning: an adjective of praise sits next to other praise, whatever it describes.

An embedding is useful **for its task** (Session 04).

S05-F07 · Nearest neighbors in the learned embedding

Six query words and their five nearest neighbors in E after training on IMDB.

`../figures/s05_f07_embedding_neighbors.png` · route: script (notebook 5.4)

Prediction before experiment

IMDB. Shuffle the words of every **test** review; the models were trained on normal text.

Bag of embeddings: what happens to its accuracy?

(Think about the formula before answering.)

Answer in notebook 5.4.

LSTM: how much does it lose?

What does the gap say about whether **order matters for sentiment**?

When order matters

Sentiment is close to a bag of words: which words appear says most of it.

Order matters for negation ("not good"), contrast ("good acting, but…"), irony, and anything with structure: dates, code, translation.

The **task** decides whether the order-aware model is worth its cost.

The limit of a single vector

Many-to-one keeps only $\mathbf{h}_T$, or a pooling of all the $\mathbf{h}_t$.

For a sentiment score, one vector is enough.

For a **translation**, that one vector must carry the whole input sentence to a model that writes another one.

Part 5 · Sequence to sequence and attention

1. Sequences as data — windows, splits and the baseline to beat
2. Recurrence and its gradients — one cell, many steps
3. Memory that lasts — LSTM and GRU
4. Text as a sequence — tokens, embeddings, order
5. **Sequence to sequence** — and the first form of attention

> Part 4: one vector summarizes the sequence. Enough to classify, not to rewrite.

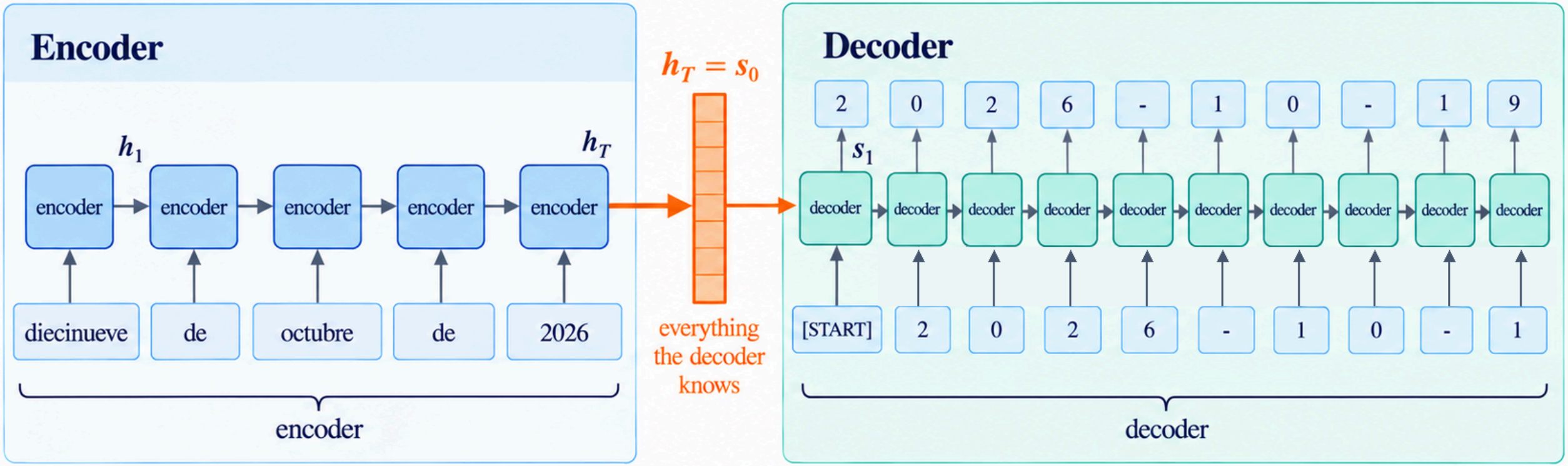
Encoder and decoder

Encoder: an RNN reads $\mathbf{x}_1, \dots, \mathbf{x}_T$ and ends in $\mathbf{h}_T$.

Decoder: another RNN starts from $\mathbf{s}_0 = \mathbf{h}_T$ and writes $y_1, \dots, y_{T'}$, one token at a time:

$$\mathbf{s}_\tau = \mathrm{RNN}(\mathbf{s}_{\tau-1}, \mathbf{e}_{y_{\tau-1}}), \qquad p(y_\tau \mid y_{<\tau}, \mathbf{x}) = \mathrm{softmax}(W_o \mathbf{s}_\tau)$$

The encoder–decoder of Session 04, now in time. Sutskever, Vinyals & Le (2014); Cho et al. (2014).



the whole input must fit in one fixed-size vector, whatever its length

Teacher forcing

Training: the decoder receives the **true** previous token $y_{\tau-1}$. All steps are trained in one pass; the loss is the cross-entropy of Session 01, summed over τ.

Inference: it receives **its own** previous prediction. Errors accumulate: exposure bias.

Decoding: greedy (take the most likely token); beam search keeps the k best partial outputs.

The bottleneck

Everything the decoder knows about the input passes through **one vector** $\mathbf{h}_T$, of fixed size, whatever the input length.

Translation quality of the plain encoder–decoder **falls with the length** of the sentence (Cho et al., 2014).

Part 2 again: the beginning of the input is many steps away from the end.

Notebook 5.5: dates inside sentences of 7–27 words. What limits the plain encoder–decoder there is the **size** of $\mathbf{h}_T$.

Look back instead of remembering

At each output step τ, the decoder **looks at every encoder state**:

$$\alpha_{\tau t} = \mathrm{softmax}_t\left(\mathrm{score}(\mathbf{s}_{\tau-1}, \mathbf{h}_t)\right), \qquad \mathbf{a}_\tau = \sum_{t=1}^{T} \alpha_{\tau t}\, \mathbf{h}_t$$

The context $\mathbf{a}_\tau$ changes at every output step and enters the decoder with $\mathbf{e}_{y_{\tau-1}}$.

Bahdanau, Cho & Bengio (2015).

Attention in motion

S05-A03 · Decoding a date with attention

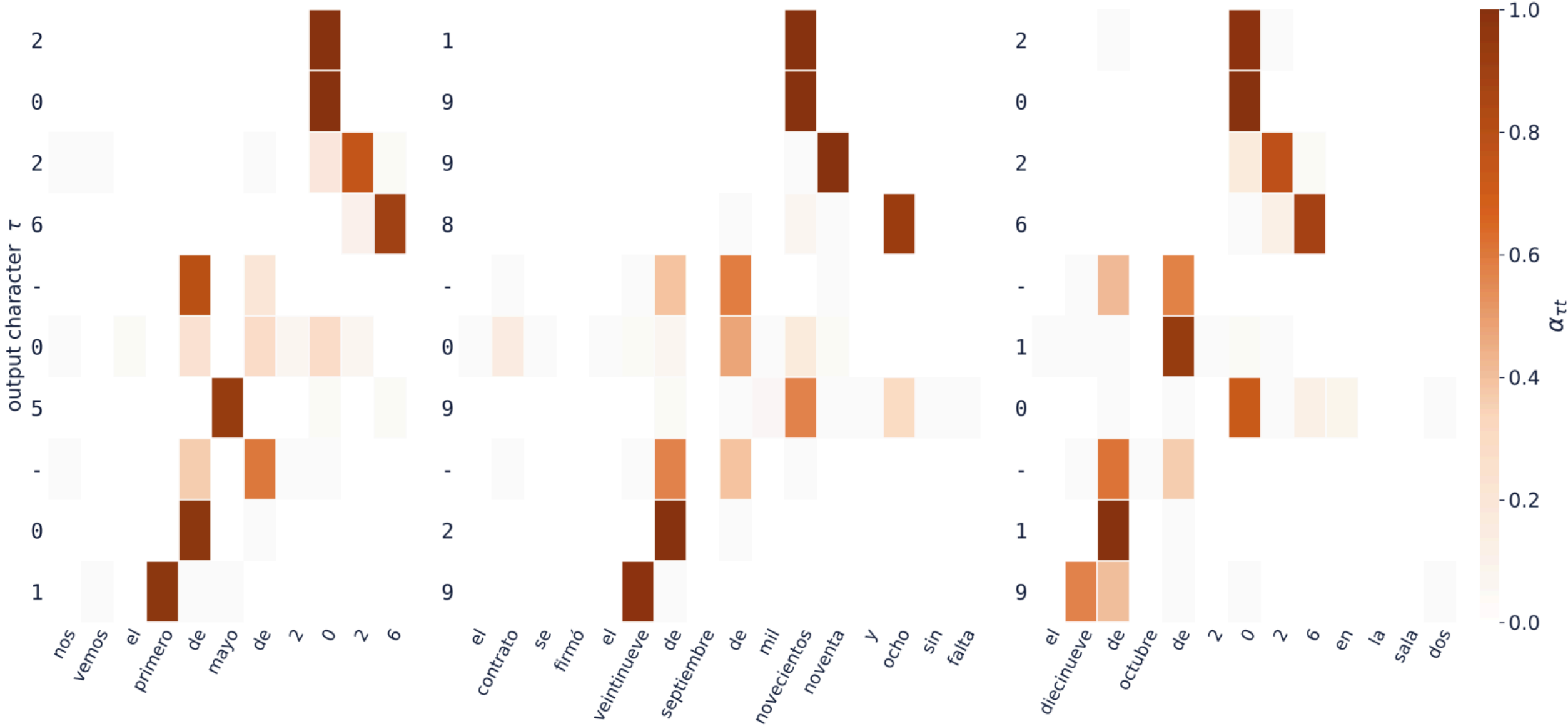
At each output character: scores over the input words, softmax into $\alpha_{\tau t}$, the weighted sum $\mathbf{a}_\tau$, the emitted character. The matrix of weights fills row by row.

`../figures/s05_a03_seq2seq_attention.mp4` · route: Manim (`escenas_s05.py`)

Score functions

Score	$\text{score}(\mathbf{s}, \mathbf{h})$	Source
additive	$\mathbf{v}^\top \tanh(W_s \mathbf{s} + W_h \mathbf{h})$	Bahdanau et al. (2015)
dot product	$\mathbf{s}^\top \mathbf{h}$	Luong et al. (2015)
scaled dot product	$\mathbf{s}^\top \mathbf{h} / \sqrt{d}$	Vaswani et al. (2017)

The scaled dot product is the one that survives in **Session 06**.



input word t · each row sums to 1 · seq2seq with additive attention, $d = 24$ (notebook 5.5)

Prediction before experiment

Spanish sentences with a date in words → ISO date. Seq2seq **without** attention, same data, same training; only the size d of $\mathbf{h}_T$ changes.

With $d = 128$ it works. With $d = 24$, which part of the date is lost first: the **year**, the **month** or the **day**?

Same $d = 24$, **with** attention. Better, worse, or the same?

Answer in notebook 5.5.

What attention changed

Notebook 5.5, exact accuracy on 6 000 test sentences:

	$d = 24$	$d = 128$
without attention	0.4 %	93 %
with attention	**100 %**	—

- without attention, what is lost is the **day**: read first, written last;
- with attention the decoder reaches **any input state directly**: path length 1 instead of T;
- the weights read as an **alignment**, with care: attention picks **states** $\mathbf{h}_t$, not words, and the state at "de" already holds the day.

Keep the attention, drop the recurrence?

The encoder is still recurrent: step t waits for step $t-1$, so training cannot run the steps in parallel.

If every step can look at every other step directly…

> …is the recurrence still needed? **Session 06.**

Practice B — Text and sequence to sequence

Worked notebooks:

- 5.4 · Text and embeddings: vocabulary from train, padding and masks, bag of embeddings vs 1-D CNN vs LSTM, shuffled words, neighbors in E · GPU
- 5.5 · Seq2seq and attention: dates in sentences → ISO, without attention at $d = 128$ and $d = 24$, with attention at $d = 24$, the attention map, how big $\mathbf{h}_T$ must be · CPU, ~7 min

> Write down for each text model: parameters, test accuracy, accuracy with shuffled words.

Integrating exercise — Bike Sharing

5.6 · Integrating exercise · hourly bike rentals, weather and calendar

- temporal split and windows **without leakage**;
- your recurrent model (LSTM or GRU), logged to **MLflow**;
- MAE at 1, 6, 12 and 24 hours ahead, against the baseline;
- `forecast_raw`: a forecast from raw hourly records that rejects those that break the contract.

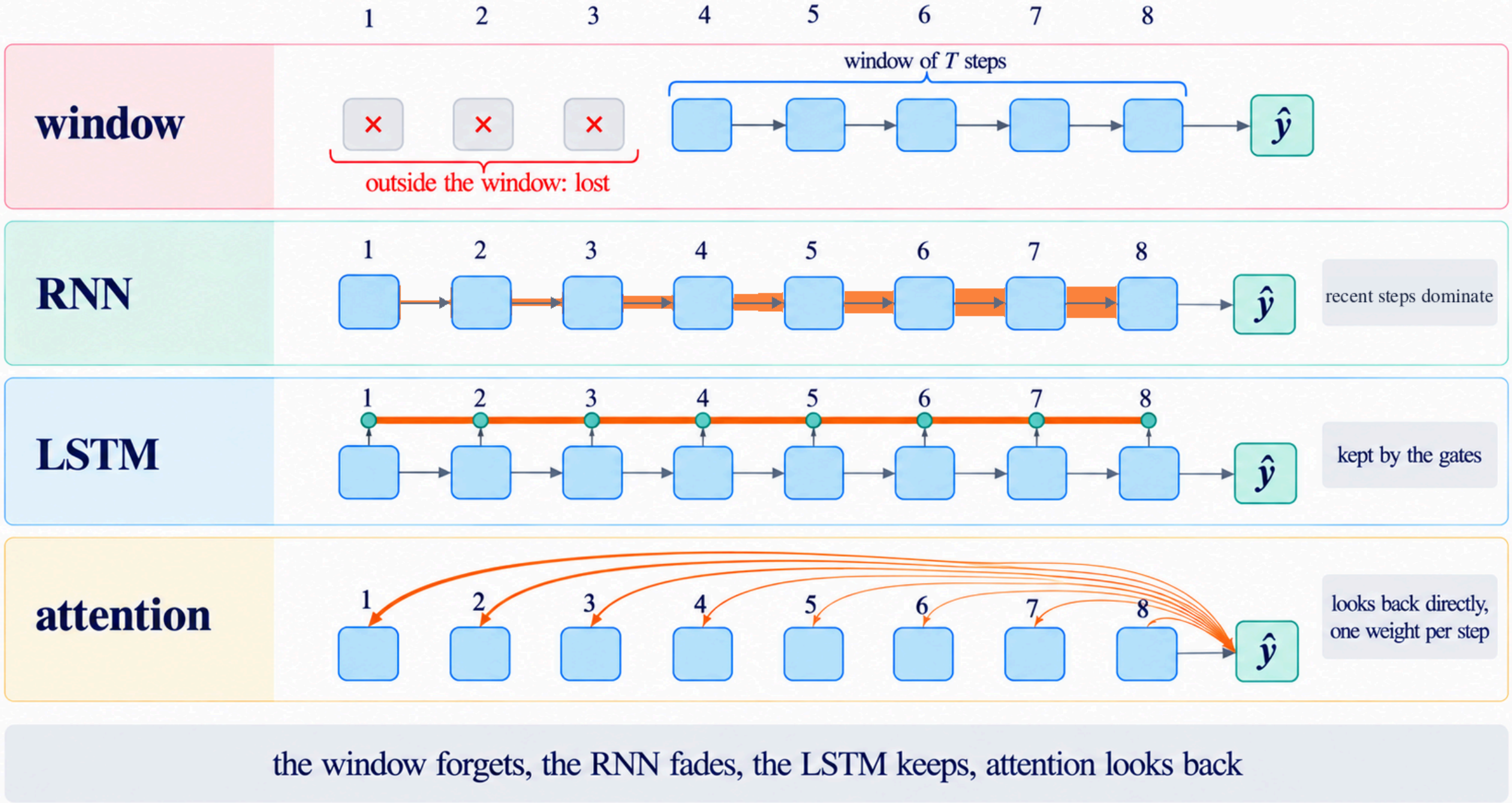
> How many hours ahead does your network still beat the obvious answer?

Choosing a sequence model

Model	Sees	Parameters grow with T?	Enough when
baseline	the last value (or season)	—	always the first row of the table
MLP on the window	T steps, no order	yes	short windows, few variables
1-D CNN	a receptive field	no	local patterns
RNN	everything, fading	no	short dependencies
LSTM / GRU	everything, kept	no	long dependencies
seq2seq + attention	every input step, at every output step	no	output is another sequence

What each choice assumes

	Assumes	Pays with
the window	the past beyond T does not matter	what happened before T
the RNN	the past fits in $\mathbf{h}_t$	the fading gradient
the LSTM	gates can learn what to keep	4× the RNN parameters, sequential training
bag of embeddings	order does not matter	negation, structure
attention	any step may matter for any output	a weight for every pair of steps


1
2
3
4
5
6
7
8
window
window of T steps
outside the window: lost
ŷ
RNN
recent steps dominate
LSTM
kept by the gates
attention
looks back directly, one weight per step
the window forgets, the RNN fades, the LSTM keeps, attention looks back

Exit question

A hospital has two years of hourly bed occupancy and wants a **48-hour** forecast.

Choose two and justify:

- a commonsense baseline with the weekly pattern;
- an LSTM with a 48-hour window;
- an LSTM with a 2-week window;
- a bidirectional LSTM over the window;
- first check that the split keeps the last months for test.